

DMB PRODTRACK

Technical Document

Architecture, data model, API, security, operations and testing

Version	1.0
Date	27 September 2026
Application release	Sprint 2-3 batch (branch feature/ sprint-2-3)
Audience	Developers, reviewers, IT administrators
Author	Deo Bernal

Practice project by DMB Websolutions. The manufacturing scenario, business rules and data are fictitious, plausible assumptions.

Contents

Overview	3
Architecture	4
Layers and projects	4
Request pipeline	5
Technology stack	7
Data model.....	8
Migrations.....	10
API reference summary	11
Scan codes	12
Security	13
Authentication	13
Authorization	13
Audit trail	14
Concurrency.....	15
Real-time updates (SignalR)	16
Logging, errors and health	17
Traveler and QR codes	18
Shop-floor PWA	19
Testing strategy	20
Continuous integration	21
Local setup and configuration	22
Prerequisites	22
Run	22
Database commands	22
Configuration keys.....	22
Deployment options (future)	24
Architecture decisions	25

Overview

DMB ProdTrack is a .NET 10 web application with two front ends served by one ASP.NET Core host:

- a **back office** built with Blazor Web App (Interactive Server rendering) for planners, supervisors, QC and admins;
- a **shop-floor PWA** built with Blazor WebAssembly, served under `/floor`, for station tablets.

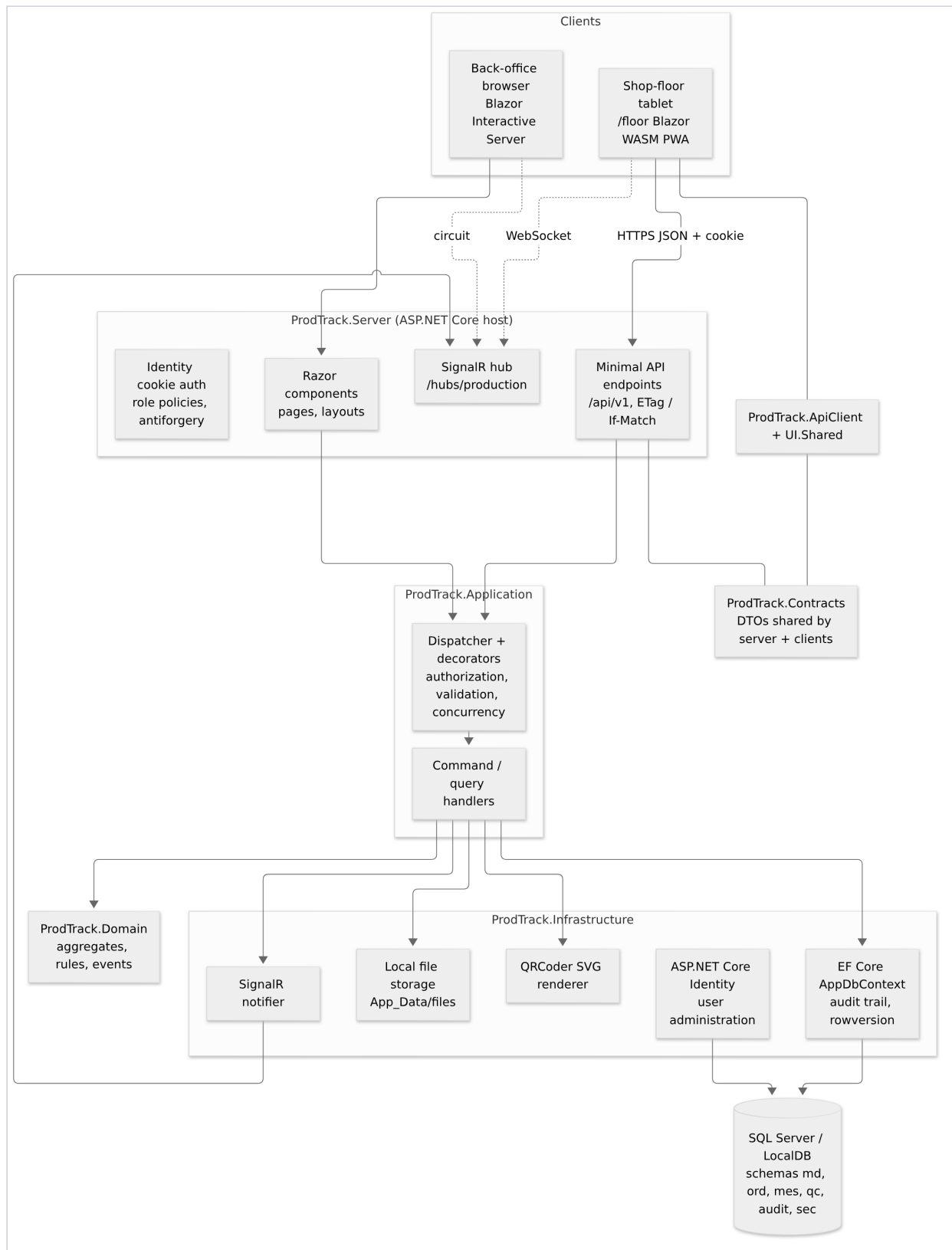
Both use the same REST API (`/api/v1`), the same cookie-based ASP.NET Core Identity sign-in and the same SignalR hub for live updates. Data lives in SQL Server (LocalDB for local development) through EF Core. This document reflects the code on branch `feature/sprint-2-3` (PR #2), 27 September 2026.

Architecture

Layers and projects

The solution (`ProdTrack.slnx`) follows Clean Architecture with a lightweight CQRS style (ADR-0001). Dependencies point inwards; architecture tests (NetArchTest) fail the build if a layer references an outer one.

Project	Layer	Responsibility
<code>ProdTrack.Domain</code>	Domain	Aggregates (WorkOrder, Operation, SalesOrder, Routing, Product, Station, ReasonCode, QcInspection, ...), invariants, state machines, <code>Result/Error</code> , domain events. No framework dependencies.
<code>ProdTrack.Application</code>	Application	Commands and queries with handlers, <code>IDispatcher</code> , decorators (authorization, validation, concurrency), FluentValidation validators, abstractions (<code>IAppDbContext</code> , <code>INotifier</code> , <code>IFileStorage</code> , <code>IQRCodeRenderer</code> , <code>IPlantClock</code> , <code>ICurrentUser</code>), role/policy definitions.
<code>ProdTrack.Infrastructure</code>	Infrastructure	EF Core <code>AppDbContext</code> and configurations, migrations, audit trail, number sequences, seeding, Identity stores and user administration, QRCode renderer, local file storage, SignalR notifier.
<code>ProdTrack.Contracts</code>	Shared	Request/response DTOs and the hub contract shared by server and clients.
<code>ProdTrack.ApiClient</code>	Shared client	Typed <code>HttpClient</code> wrapper used by the PWA (antiforgery-aware).
<code>ProdTrack.UI.Shared</code>	Shared UI	Razor components shared between back office and PWA.
<code>ProdTrack.ShopFloor</code>	Presentation	Blazor WebAssembly PWA (<code>/floor</code>): station picker, queue, scan box, operation page, SignalR client.
<code>ProdTrack.Server</code>	Presentation / host	Minimal API endpoints, SignalR hub, Blazor Server pages, Identity UI (sign-in, change password), security setup, Serilog, health checks, hosting of the PWA.



Architecture: projects, layers and runtime connections

Request pipeline

1. Endpoint (Minimal API) or Razor component builds a command/query from the request.

2. `IDispatcher` resolves the handler, wrapped by decorators (outermost first):
Authorization (policy from `[RequiresPolicy]`) → **Validation** (`FluentValidation`) → **Concurrency** (maps `DbUpdateConcurrencyException` to `Concurrency.Conflict`) → handler.
3. The handler loads aggregates through `IAppDbContext`, calls domain methods that return `Result`, and saves.
4. `SaveChanges` writes audit entries in the same transaction and dispatches domain events after commit (for example to the SignalR notifier).
5. Endpoints map `Result` errors to `ProblemDetails`: `Validation` → 400, `NotFound` → 404, `Conflict` → 409, `BusinessRule` → 422, `Forbidden` → 403, `Concurrency.StaleVersion` → 412, `Concurrency.Conflict` → 409.

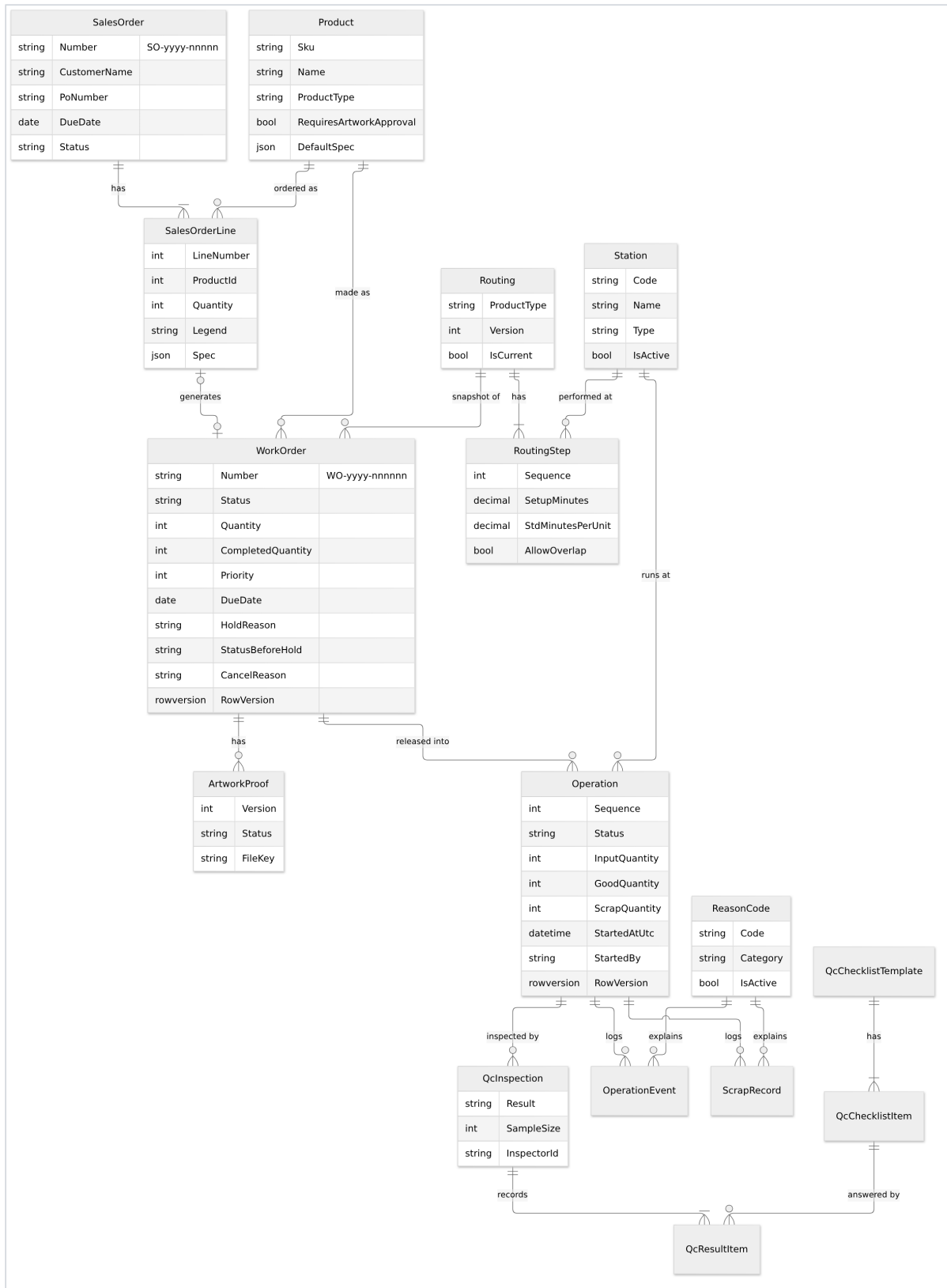
Technology stack

Area	Technology	Version
Runtime / SDK	.NET, C#	SDK 10.0.401 (global.json, roll-forward latestFeature)
Web host	ASP.NET Core (Minimal APIs, Blazor Web App, SignalR)	10.0
Shop floor	Blazor WebAssembly PWA, SignalR client	10.0
Data access	EF Core (SqlServer provider; Sqlite for tests)	10.0
Database	SQL Server LocalDB MSSQLLocalDB (local); SQL Server / MonsterASP MSSQL (future hosting)	-
Identity	ASP.NET Core Identity (EF stores), cookie auth	10.0
Validation	FluentValidation	12.1.1
DI scanning / decorators	Scrutor	7.0.0
QR codes	QRCoder (MIT) - SVG output	1.8.0
Logging	Serilog.AspNetCore, Serilog.Sinks.File, Serilog.Formatting.Compact	10.0.0 / 7.0.0 / 3.0.0
API docs	Microsoft.AspNetCore.OpenApi + Swashbuckle SwaggerUI (Development)	10.0 / 10.2.3
Unit/integration tests	xUnit, AwesomeAssertions, NSubstitute, Microsoft.AspNetCore.Mvc.Testing, TimeProvider.Testing	2.9.3 / 9.6.0 / 6.2.0 / 10.0 / 10.10.0
Architecture tests	NetArchTest.Rules	1.3.2
E2E tests	Microsoft.Playwright (Chromium)	1.63.0
Coverage	coverlet.collector, ReportGenerator (local tool)	10.0.1
CI	GitHub Actions (ubuntu-latest)	-

Package versions are pinned centrally in `Directory.Packages.props`. Commercially licensed packages (FluentAssertions v8+, MediatR v13+, AutoMapper) are deliberately avoided.

Data model

EF Core maps the domain to SQL Server schemas: `md` (master data), `ord` (orders and work orders), `mes` (execution), `qc` (quality), `audit` and `sec` (Identity and data-protection keys). Specifications are stored as JSON columns. Aggregates carry a SQL Server `rowversion` column used for optimistic concurrency.



Entity relationship diagram (main entities)

Entity	Schema	Notes
Product	md	SKU, type (PipeMarker, ValveTag, SafetySign, Label), RequiresArtworkApproval , default spec (JSON)

Entity	Schema	Notes
Station	md	Code, name, type (Prepress, Printing, Laminating, Engraving, Cutting, Inspection, Packing), active flag
ReasonCode	md	Code, description, category (Scrap, Pause, Downtime, Hold, Skip)
ColorScheme, SignalWord	md	ASME A13.1 and ANSI Z535 reference data
Routing, RoutingStep	md	Versioned per product type; one current version
SalesOrder, SalesOrderLine	ord	Number S0-yyyy-nnnnn, status Open/Closed/Cancelled; line spec snapshot
WorkOrder	ord	Number W0-yyyy-nnnnnn, optional SalesOrderLineId, spec snapshot, status, hold reason, StatusBeforeHold, CancelReason
ArtworkProof	ord	Versioned proof files with status Pending/Approved/Rejected/Superseded
Operation	mes	Sequence, station, status, input/good/scrap quantity, standard times, StartedAtUtc, StartedBy, rowversion
OperationEvent	mes	Start/pause/resume/complete events with reason, user, device and idempotency RequestId
ScrapRecord	mes	Quantity, reason, note, user, time
QcChecklistTemplate, QcChecklistItem	qc	Per product type; Pass/Fail or Measured items with min/max/unit
QcInspection, QcResultItem	qc	Result Passed/Failed, sample size, inspector, per-item results
AuditEntry	audit	Entity, key, action, changes JSON (old → new), user, correlation id, UTC time
AppUser (+ roles)	sec	Identity user with display name, employee number, badge number, home station, IsActive, MustChangePassword
NumberSequence	ord	Per-kind, per-year counters with rowversion

Migrations

Migration	Content
20260927124814_InitialCreate	All schemas and tables of Sprint 0-1 (master data, orders, execution, QC, audit, Identity)
20260927143317_Sprint23Execution	WorkOrder.CancelReason, WorkOrder.StatusBeforeHold, Operation.StartedBy and related indexes

`dotnet ef database update` creates a clean database from these migrations. The deploy workflow (future) applies an idempotent script generated by CI instead of migrating at startup.

API reference summary

All endpoints are under `/api/v1`, require an authenticated user (fallback policy) and return `ProblemDetails` with a `traceId` on errors. Unsafe requests from cookie-authenticated browsers must send the antiforgery token in `X-XSRF-TOKEN` (fetch it from `GET /api/v1/antiforgery`). Swagger UI is available at `/swagger` in Development.

Area	Endpoints	Policy
Me	<code>GET /me</code>	ReadAll
Dashboard	<code>GET /dashboard/wip</code>	ReadAll
Reference	<code>GET /reference/{list}</code> (color-schemes, signal-words, product-types, station-types, reason-categories)	ReadAll
Products	<code>GET /products</code> , <code>GET /products/{id}</code> (ETag), <code>POST /products</code> , <code>PUT /products/{id}</code> (If-Match)	ReadAll / ManageProductsAndRoutings
Stations	<code>GET /stations</code> , <code>POST /stations</code> , <code>PUT /stations/{id}</code> (If-Match), <code>DELETE /stations/{id}</code> (deactivate)	ReadAll / ManageMasterData
Reason codes	<code>GET /reason-codes</code> , <code>POST /reason-codes</code> , <code>PUT /reason-codes/{id}</code> (If-Match)	ReadAll / ManageMasterData
Routings	<code>GET /routings</code> , <code>GET /routings/{id}</code> , <code>POST /routings</code> (new version)	ReadAll / ManageProductsAndRoutings
Sales orders	<code>GET /sales-orders</code> , <code>GET /sales-orders/{id}</code> (ETag), <code>POST /sales-orders</code> , <code>PUT /sales-orders/{id}</code> (If-Match), <code>POST /sales-orders/{id}/cancel</code> , <code>POST /sales-orders/{id}/work-orders</code>	ReadAll / PlanWorkOrders
Work orders	<code>GET /work-orders</code> , <code>GET /work-orders/{id}</code> (ETag), <code>GET /work-orders/by-number/{number}</code> , <code>POST /work-orders</code> , <code>PUT /work-orders/{id}</code> (If-Match), <code>DELETE /work-orders/{id}</code> , <code>POST /work-orders/{id}/release</code>	ReadAll / PlanWorkOrders
Work order control	<code>POST /work-orders/{id}/hold</code> , <code>/resume</code> , <code>/cancel</code> (all If-Match), <code>GET /work-orders/{id}/traveler</code>	ControlWorkOrders
Artwork	<code>POST /work-orders/{id}/artwork</code> , <code>GET .../artwork/{version}</code> , <code>POST .../artwork/{version}/approve</code> , <code>.../reject</code>	UploadArtwork / ReadAll / ApproveArtwork
Floor	<code>GET /stations/{code}/queue</code> , <code>GET /scan/{code}</code>	ReadAll / ExecuteOperations
Operations	<code>GET /operations/{id}</code> , <code>POST /operations/{id}/start</code> , <code>/pause</code> , <code>/resume</code> , <code>/complete</code> , <code>/scrap</code>	ReadAll / ExecuteOperations
QC	<code>GET /qc/templates</code> , <code>POST /operations/{id}/inspection</code>	ReadAll / RecordInspections
Users	<code>GET /users</code> , <code>GET /users/{id}</code> (ETag), <code>POST /users</code> , <code>PUT /users/{id}</code> (If-Match), <code>POST /users/{id}/active</code> , <code>POST /users/{id}/reset-password</code>	ManageUsers

Other routes: `/hubs/production` (SignalR), `/health`, `/health/live`, `/health/ready`, `/account/login`, `/account/logout`, `/account/change-password`, `/floor/` (PWA).

Scan codes

The traveler QR codes carry `W0:{number}` for the work order and `OP:{number}:{sequence}` for an operation (ADR-0007). `GET /scan/{code}` also accepts a bare work order number (typed by hand) and returns the target operation or a 404 ProblemDetails with a readable message.

Security

Authentication

- ASP.NET Core Identity with EF stores; accounts are created by administrators only (no self-registration, no email).
- Cookie `__Host-ProdTrack`: HttpOnly, Secure, SameSite=Lax, 12-hour sliding expiration. API calls receive 401/403 ProblemDetails instead of redirects.
- Password policy: minimum 12 characters; lockout after 5 failures for 15 minutes; `MustChangePassword` forces a change after creation or reset. Disabled users are rejected at sign-in and at the next security-stamp validation (30 minutes).
- Bootstrap admin: created on an empty database from `Auth:BootstrapAdmin` (Development default `admin@prodtrack.local` / `ChangeMe!Dev2026`; outside Development it must come from host configuration).
- Optional `Auth:Mode=Dev` user picker for local development only (refused outside Development).

Authorization

Roles: Admin, Planner, Supervisor, Operator, QC, Viewer. Policies are defined once in `Policies.Definitions` and used by endpoints, Razor `AuthorizeView` and the application authorization decorator.

Policy	Roles
ReadAll	all roles
ManageMasterData, ManageUsers	Admin
ManageProductsAndRoutings, PlanWorkOrders, ApproveArtwork	Admin, Planner
ControlWorkOrders	Admin, Planner, Supervisor
ExecuteOperations	Admin, Supervisor, Operator, QC
RecordInspections	Admin, QC
UploadArtwork	Admin, Planner, Operator
ViewAudit	Admin, Supervisor, QC

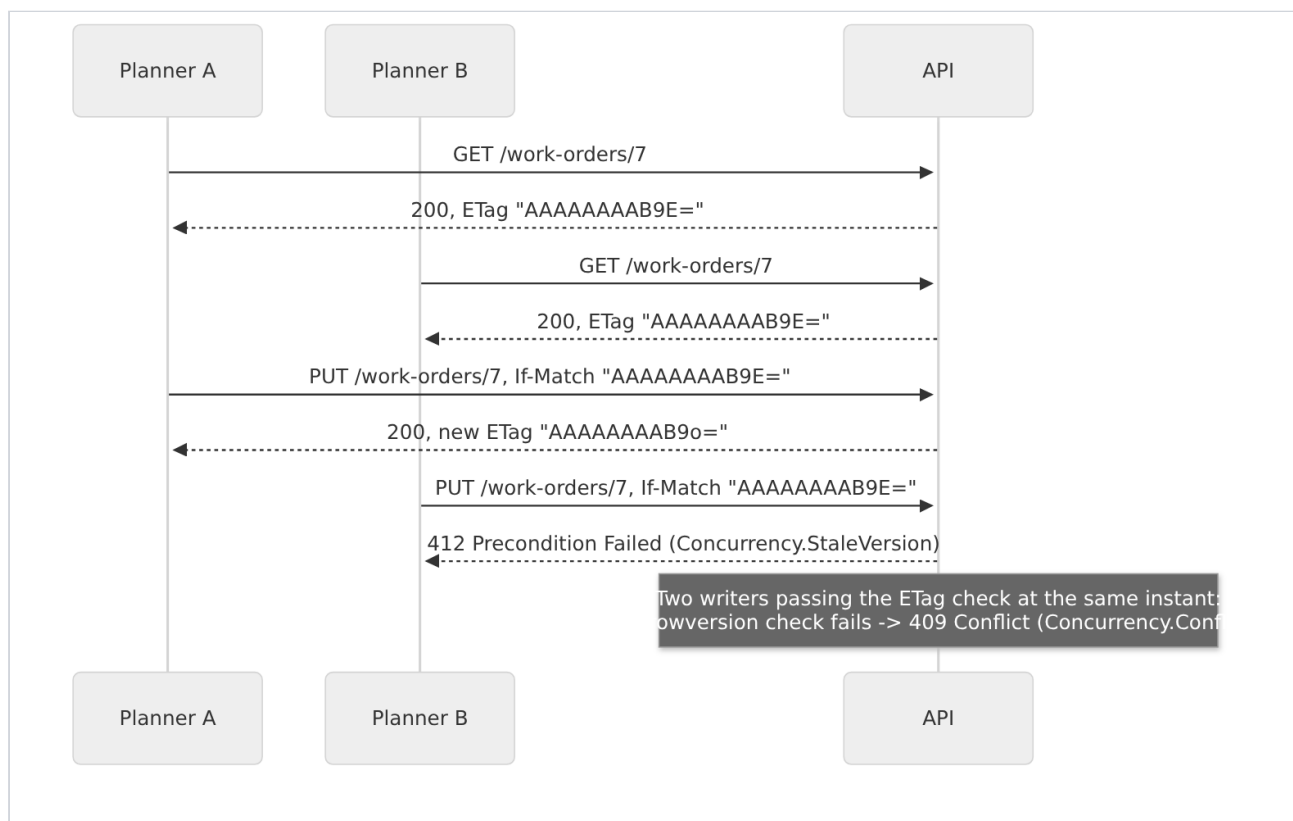
Antiforgery protection applies to forms and to cookie-authenticated API calls (X-XSRF-TOKEN).

Audit trail

`AppDbContext.SaveChanges` captures added, modified and deleted entities from the change tracker and writes `AuditEntry` rows in the **same transaction** (ADR-0011): entity name, key, action, a JSON document of changed fields (old → new), user id, correlation id and UTC timestamp. Secrets and noise (password hashes, security stamps, rowversions, normalized names) are excluded. User administration actions are audited too.

Concurrency

- Every aggregate root has a `rowversion` column configured as an EF concurrency token (emulated for SQLite in tests).
- GET of a single resource returns an ETag (base64 rowversion; the Identity concurrency stamp for users).
- PUT and state-changing POST endpoints accept If-Match:
- missing or * → last write wins (backwards compatible);
- stale or malformed → **412 Precondition Failed**, code `Concurrency.StaleVersion`;
- a concurrent write between the check and the save (EF `DbUpdateConcurrencyException`) → **409 Conflict**, code `Concurrency.Conflict`, via the `ConcurrencyCommandDecorator`.
- Blazor edit pages keep the version they loaded and show "Someone else changed this ... Reload latest" on 409/412.
- Floor commands accept an optional `RequestId` so retried taps are idempotent.

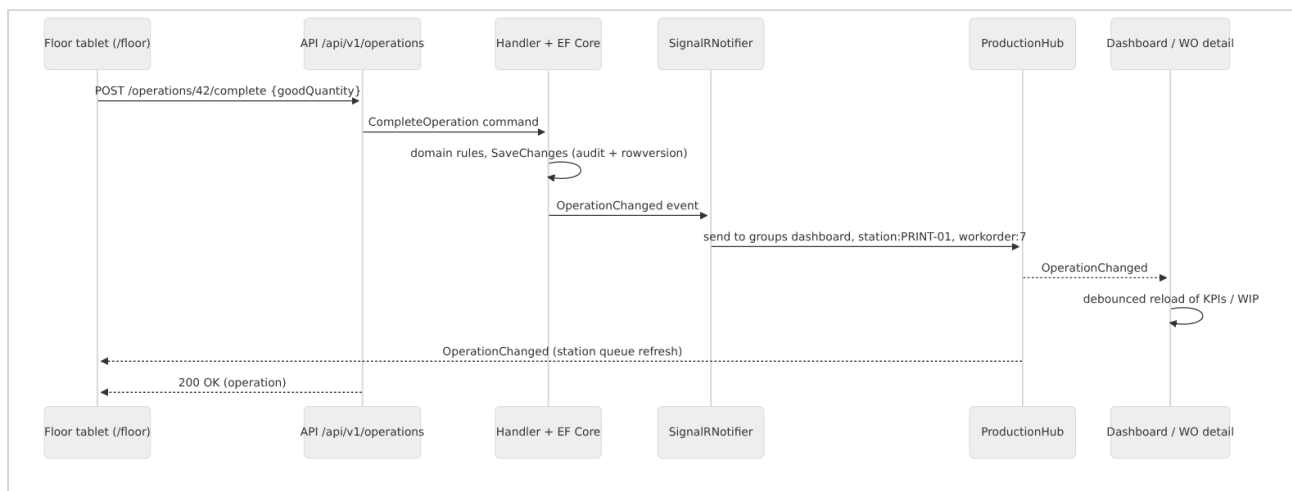


If-Match flow

Real-time updates (SignalR)

The strongly typed hub `ProductionHub` at `/hubs/production` pushes server-to-client messages only (ADR-0006): `WorkOrderChanged` and `OperationChanged`. Groups: `dashboard`, `station:{code}` and `workorder:{id}`.

- `SignalRNotifier` (Infrastructure) sends events after the database commit.
- Back office: an `IRealtimeFeed` singleton relays hub events to Blazor Server components; the `LiveRefresh` component debounces bursts and reloads the dashboard or work order detail. The former 30-second polling was removed.
- Shop floor: `LiveUpdates` uses the SignalR .NET client in WebAssembly with automatic reconnect and refreshes the station queue and operation page.



Live update sequence

Logging, errors and health

- **Serilog** writes compact JSON rolling files to `App_Data/logs/prodtrack-.json` (and the console in Development), with request logging and a correlation id (`X-Correlation-Id`) on every request and audit entry.
- A global exception handler returns `ProblemDetails` with `traceId` for API calls and a friendly error page for the UI.
- Health endpoints: `/health/live` (process), `/health/ready` and `/health` (database check). Anonymous access.

Traveler and QR codes

GET `/work-orders/{id}/traveler` returns the work order, operations and QR codes rendered by **QRCoder** as SVG strings (`IQrCodeRenderer`). The Blazor page `/work-orders/{id}/traveler` uses a print layout without navigation and inlines the SVGs, so printing or saving as PDF from the browser needs no server-side PDF engine.

Shop-floor PWA

- Blazor WebAssembly app hosted by the server under `/floor` (same origin, same cookie), installable via a web manifest and service worker.
- `StationPreference` stores the chosen station in `localStorage`.
- `ScanBox` accepts keyboard-wedge scanners and typed codes, and uses the browser `BarcodeDetector` API through `wwwroot/js/floor.js` for camera scanning where available (feedback tones included).
- `ApiClient` adds the antiforgery header to unsafe requests.
- WebAssembly hot reload is disabled for this project (`WasmEnableHotReload=false`) because its reload module breaks the server-hosted pages.

Testing strategy

Project	Type	What it covers	Tests
ProdTrack.Domain.Tests	Unit	Aggregates, state machines, quantity rules, QC evaluation, scan code parsing	56
ProdTrack.Application.Tests	Unit	Handlers and decorators with SQLite in-memory and fakes (sales orders, hold/cancel, execution, QC, concurrency)	56
ProdTrack.Infrastructure.Tests	Integration (component)	Audit trail, number sequences, QR renderer, plant clock, local file storage	13
ProdTrack.ArchitectureTests	Architecture	Layer dependency rules (NetArchTest)	6
ProdTrack.Server.IntegrationTests	Integration	Full HTTP pipeline via WebApplicationFactory + SQLite in-memory: every endpoint, role matrix, ETag/If-Match 412/409, ProblemDetails, health	110
ProdTrack.E2E.Tests	End-to-end	Playwright/Chromium against an in-process Kestrel host: sign-in with forced password change, create and release a work order, run an operation on /floor, scan box	2 (+1 opt-in screenshot capture)
Total			243 (+1)

- Tests need neither SQL Server nor Docker: SQLite in-memory replaces SQL Server (ADR-0010), with rowversion emulation for concurrency tests.
- Tests are tagged with `Category` (Unit, Integration, E2E) and `Story` (PT-xxx) traits.
- E2E: the first run installs Chromium user-locally (`%LOCALAPPDATA%\ms-playwright` on Windows, `~/.cache/ms-playwright` on Linux). `PRODTRACK_E2E_HEADED=1` shows the browser; failure screenshots go to `%TEMP%/prodtrack-e2e-failures`. The docs screenshot capture runs only with `PRODTRACK_DOCS_SCREENSHOTS=<dir>`.
- Quality gates: `dotnet format --verify-no-changes`, `dotnet build -warnaserror` (0 warnings), all tests green.

Continuous integration

`.github/workflows/ci.yml` runs on pull requests to `main`, pushes to `main` and manual dispatch, on GitHub-hosted `ubuntu-latest`. It needs **no secrets**.

Job	Steps
<code>build-test</code>	Restore tools and packages, format check, Release build with <code>-warnaserror</code> , unit tests, integration tests (coverage + trx), coverage summary, vulnerable-package check; on <code>main</code> also publishes the <code>win-x86</code> framework-dependent app and an idempotent migration script as the <code>prodtrack-drop</code> artifact
<code>e2e</code>	Build the E2E project, install Chromium with OS dependencies on the runner, run <code>Category=E2E</code> , upload trx files and failure screenshots on failure

`deploy.yml` (manual, MonsterASP Web Deploy) and `ops.yml` (weekly checks) exist from the scaffold but have not been run; they require secrets that are intentionally not configured.

Local setup and configuration

Prerequisites

- .NET SDK 10.0.401 or later feature band; SQL Server Express LocalDB (installed with Visual Studio 2026).
- Visual Studio 2026 (or the `dotnet` CLI). No Docker needed.

Run

```
dotnet tool restore
dotnet build ProdTrack.slnx
dotnet run --project src/ProdTrack.Server --launch-profile https
# https://localhost:5001 (back office) https://localhost:5001/floor/ (shop floor)
```

In Visual Studio 2026 open `ProdTrack.slnx`, set **ProdTrack.Server** as startup project, choose the **https** profile and press F5. In Development the app applies pending migrations and seeds reference and demo data on startup.

Database commands

```
dotnet ef database update -p src/ProdTrack.Infrastructure -s src/ProdTrack.Server #
create / upgrade
dotnet ef database drop -f -p src/ProdTrack.Infrastructure -s src/ProdTrack.Server #
reset (then run again)
dotnet ef migrations add <Name> -p src/ProdTrack.Infrastructure -s src/ProdTrack.Server
-o Persistence/Migrations
```

Configuration keys

Key	Default (Development)	Purpose
ConnectionStrings:ProdTrack	Server=(localdb) \MSSQLLocalDB;Database=ProdTrack;...	SQL Server connection
Database:MigrateOnStartup	true (false in base settings)	Apply EF migrations at startup
Database:SeedDemoData	true	Seed demo products
Auth:Mode	Identity (Dev = user picker, Development only)	Sign-in mode
Auth:BootstrapAdmin:Email / Password	admin@prodtrack.local / ChangeMe! Dev2026	First administrator on an empty database
Storage:Local:RootPath	App_Data/files	Artwork files
LogFile:Enabled / LogFile:Path	true / App_Data/logs/prodtrack-.json	Serilog file sink

Key	Default (Development)	Purpose
Plant:TimeZone	Asia/Manila	Plant-local dates (late, today KPIs)
App:PublicBaseUrl , App:UseForwardedHeaders	empty / false	Reverse-proxy hosting

Secrets for local overrides go into `dotnet user-secrets` (project `ProdTrack.Server`), never into the repository.

Deployment options (future)

No deployment has been performed. The documented target is:

1. **MonsterASP.NET free plan** (US\$0): one IIS site (planned `https://dmb-prodtrack.runasp.net`) and a 1 GB MSSQL database. CI publishes a `win-x86` framework-dependent build; `deploy.yml` uses Web Deploy (FTP alternative), takes the site offline with `app_offline.htm`, applies the idempotent migration script and brings it back. Free-plan limits (256 MB RAM, sleep after 30 minutes idle, no email, manual HTTPS renewal) are handled by design: no background jobs, reconnect-aware SignalR clients, no email features.
2. **Azure DevOps (phase 2)**: mirror or migrate Boards, Repos and Pipelines (YAML equivalent of the GitHub workflows, environments with approvals), as documented in `docs/04-azure-devops-setup.md`.

Optional alternatives (Azure App Service + Azure SQL, Google Cloud Run) are described in `docs/03`.

Architecture decisions

ADR	Decision
0001	Clean Architecture with CQRS-lite (own dispatcher, no MediatR)
0002	Blazor Server back office and WebAssembly PWA in a single host
0003	ASP.NET Core Identity with admin-created accounts
0004	MonsterASP.NET free hosting
0005	SQL Server with EF Core migrations (idempotent scripts in deployment)
0006	Minimal APIs for commands, SignalR for push only
0007	QR payload format W0: / OP:
0008	Custom domain deferred
0009	GitHub Actions with Web Deploy
0010	SQLite in-memory for tests
0011	Audit in SaveChanges and database number sequences